

My Advanced Pong Project: Design & Implementation

1. The Core Architecture

For this project, I designed the program following the classic **Game Loop** pattern, widely used in professional game engines:

1. **Initialization** – Setting up the "Big Three":
 2. **Window** for rendering.
 3. **Audio Device** for sounds and music.
 4. **RAM Structures** for storing game objects like players and the ball.
 5. **Update (The Brain)** – Core logic processing:
 6. Handling **user input**.
 7. Moving **physics objects**.
 8. Checking **collisions** between the ball and players or walls.
 9. **Draw (The Body)** – Rendering the current **state of the world** to the GPU back-buffer.
 10. **De-Initialization** – Releasing all **hardware handles** and cleaning up memory, maintaining the **Symmetry of C**.
-

2. Data Structures (The "Why")

I structured my objects using **structs** with a **Data-Oriented Design** approach:

- **Player Struct** – Combines the **physical body** (`Rectangle`) and behavioral traits like `direction` and `speed`.
- **Ball Struct** – Includes **acceleration** and **direction vector** (`Vector2`) for movement at any angle.

Using structs keeps my program **modular, clean, and scalable**.

3. Key Functions & Logic

A. Delta Time (`dt`) Implementation

```
ball.body.x += ball.direction.x * ball.speed * dt;
```

- **Logic:** Using `GetFrameTime()` for `dt`. - **Reason:** Ensures consistent movement across different monitor refresh rates by moving in **pixels per second** instead of **pixels per frame**.

B. Vector Normalization (`NormalizeDirection`)

- **Logic:** Calculate vector length: $len = \sqrt{x^2 + y^2}$
 - **Reason:** Diagonal movement would otherwise be faster; normalizing ensures **constant speed** regardless of angle.
-

C. Angular Reflection (`AngularMovement`)

```
direction.y = (ballCenterY - playerCenterY) / (player.height / 2.0f);
```

- **Effect:** Hitting different parts of the paddle changes the ball's vertical direction: - **Top** → **negative** (ball moves up) - **Center** → **0** (ball moves straight)

This allows **skilled shots** and **controlled gameplay**.

D. Progressive Difficulty (`UpdateBall`)

```
ball->speed *= ball->acceleration;
```

- **Mechanism:** Increment acceleration by 0.05 per paddle hit. - **Effect:** Game starts easy but reaches high speeds (~700 pixels/sec), testing **reaction time**.

4. Special Checks & Boundary Handling

A. Collision Correction

```
ball->body.x = player2.x - ball->body.width;
```

- Prevents the ball from sinking into the paddle at high speed. - Ensures accurate **collision detection**.

B. Score Synchronization

```
bool scored = CalculateScore(...);
```

- Uses a **boolean latch** to prevent multiple score increments per frame. - Score only updates once until **SPACE reset**, ensuring **reliable scoring**.

5. Resource Management

Implemented full **Symmetry of C**: - **Initialization**: `InitWindow()`, `InitAudioDevice()`, `LoadMusicStream()` - **Cleanup**: `UnloadMusicStream()`, `UnloadSound()`, `CloseAudioDevice()`, `CloseWindow()`

All resources are released, guaranteeing **zero memory leaks** and clean **RAM/VRAM return**.